Laporan UTS Graf Sains Data

Oct 23, 2026 · @arsa

Identitas

Keterangan	Isi
Nama	Andimas Rahmad Sanjaya
NIM	23031554093
Mata kuliah	Graf Sains Data (UTS)
Dataset	RetailRocket e-commerce (`events.csv`)

Dataset yang digunakan adalah `events.csv` dari RetailRocket dengan jumlah 2.756.101 baris data interaksi pada rentang 3 Mei 2015 sampai 18 September 2015. Data tersebut diimpor ke Neo4j dengan skema graf sebagai berikut.

```
(:Visitor {visitorId})-[:VIEWED | :ADDED_TO_CART | :BOUGHT {timestamp,
transactionId}]->(:Item {itemId})
```

Sebelum impor, dibuat constraint unik pada `Visitor.visitorId` dan `Item.itemId`. Proses impor dilakukan per 1.000 baris menggunakan `UNWIND` dan `MERGE`, dan setelah selesai database berisi 1.407.580 simpul Visitor.

2.1.1 Penentuan Dua Periode Waktu

Karena data RetailRocket hanya mencakup 3 Mei 2015 sampai 18 September 2015, kedua periode dipilih di dalam rentang tersebut. Periode training (P1) adalah 12 Mei 2015 sampai 31 Juli 2015, sedangkan periode testing (P2) adalah 1 Agustus 2015 sampai 21 Agustus 2015. Kedua periode dibuat berurutan dan tidak saling tumpang tindih, sehingga model dibangun dari data masa lalu lalu diuji pada tiga minggu setelahnya.

Fold	Mulai (UTC)	Selesai (UTC)	Mulai (epoch ms)	Selesai (epoch ms)	Kegunaan
P1 (training)	2015-05-12 00:00:00	2015-07-31 23:59:59.999	1431388800000	1438387199999	VSM, proyeksi, similarity, model KNN
P2 (testing)	2015-08-01 00:00:00	2015-08-21 23:59:59.999	1438387200000	1440201599999	data uji untuk Hit@5 dan NDCG@5

Selain itu, setiap jenis interaksi diberi bobot berbeda sesuai tingkat ketertarikan pengguna, yaitu `view` = 1, `addtocart` = 3, dan `transaction` = 5.

2.1.2 Filter Timestamp dan Penyimpanan CSV

Filter dilakukan pada kolom `timestamp` yang berformat epoch milidetik. Baris yang masuk P1 atau P2 diberi label `fold` dan `weight`. Baris yang duplikat persis (visitor, item, event, dan timestamp sama) juga dihapus, karena di Neo4j baris seperti itu akan tergabung menjadi satu relasi oleh `MERGE`. Hasilnya disimpan ke file `events_filtered_P1_P2.csv`.

```
in_p1 = events["timestamp"].between(*P1_MS)
in_p2 = events["timestamp"].between(*P2_MS)

filtered = events.loc[in_p1 | in_p2].copy()
filtered["fold"] = np.where(in_p1[in_p1 | in_p2], "P1", "P2")
filtered["weight"] = filtered["event"].map(EVENT_WEIGHT)
filtered["event_time"] = pd.to_datetime(filtered["timestamp"], unit="ms",
utc=True)
filtered = (filtered.drop_duplicates(["visitorid", "itemid", "event",
"timestamp"])
            .sort_values("timestamp").reset_index(drop=True))
filtered.to_csv("events_filtered_P1_P2.csv", index=False)
```

Dari 2.756.101 baris awal, tersisa 2.113.691 baris setelah difilter. Rinciannya per fold adalah sebagai berikut.

Fold	Interaksi	Visitor unik	Item unik	view	addtocart	transaction
P1	1.735.130	884.614	192.603	1.678.507	42.437	14.186
P2	378.561	215.792	93.162	365.419	10.004	3.138

Interaksi pertama pada P1 tercatat pukul 00:00:03 tanggal 12 Mei 2015 dan interaksi terakhir pada P2 tercatat pukul 23:59:59 tanggal 21 Agustus 2015, sehingga batas filter sudah sesuai dengan periode yang ditentukan.

2.1.3 Komposisi Subgraph

Agar subgraph di Neo4j sama dengan data CSV, filter timestamp yang sama diterapkan langsung pada relasi di Neo4j. Relasi yang masuk P1 atau P2 diberi properti `fold` dan `weight`, sedangkan relasi di luar kedua periode tidak diberi `fold`. Dengan begitu, subgraph adalah semua relasi yang memiliki `fold` beserta simpul di kedua ujungnya.

```
MATCH ()-[r:VIEWED|ADDED_TO_CART|BOUGHT]->()
CALL (r) {
  SET r.fold = CASE
        WHEN r.timestamp >= $p1_start AND r.timestamp <= $p1_end THEN 'P1'
        WHEN r.timestamp >= $p2_start AND r.timestamp <= $p2_end THEN 'P2'
        ELSE null END,
      r.weight = CASE type(r) WHEN 'VIEWED' THEN 1 WHEN 'ADDED_TO_CART' THEN
3 ELSE 5 END
} IN TRANSACTIONS OF 50000 ROWS
```

Bagian `MATCH` mengambil semua relasi interaksi. Karena jumlah relasinya sangat banyak, update dijalankan dengan `CALL (r) { ... } IN TRANSACTIONS OF 50000 ROWS` supaya diproses per 50.000 relasi dan memori transaksi tidak penuh. Ekspresi `CASE` dipakai untuk menentukan fold berdasarkan timestamp (bernilai `null` jika di luar periode) dan menentukan bobot berdasarkan jenis relasi. Proses ini selesai dalam 31,2 detik.

Setelah itu, komposisi subgraph diambil dengan tiga query berikut yang dijalankan melalui Python API (`driver.session().run(query).data()`).

```
// (a) jenis dan jumlah simpul pada subgraph
MATCH (n)-[r:VIEWED|ADDED_TO_CART|BOUGHT]-()
WHERE r.fold IS NOT NULL
WITH DISTINCT n
RETURN labels(n)[0] AS jenis_simpul, count(*) AS jumlah
ORDER BY jumlah DESC

// (b) jumlah simpul dan relasi per fold
MATCH (v:Visitor)-[r:VIEWED|ADDED_TO_CART|BOUGHT]->(i:Item)
WHERE r.fold IS NOT NULL
RETURN r.fold AS fold, count(DISTINCT v) AS visitor, count(DISTINCT i) AS
item, count(r) AS relasi
ORDER BY fold

// (c) jumlah relasi per jenis interaksi per fold
MATCH ()-[r:VIEWED|ADDED_TO_CART|BOUGHT]->()
WHERE r.fold IS NOT NULL
RETURN r.fold AS fold, type(r) AS jenis_relasi, count(*) AS jumlah
ORDER BY fold, jumlah DESC
```

Query (a) mencari setiap simpul yang memiliki minimal satu relasi dengan `fold` tanpa memperhatikan arah relasi. `WITH DISTINCT n` digunakan agar setiap simpul hanya dihitung satu kali walaupun memiliki banyak relasi, kemudian hasilnya dikelompokkan berdasarkan label. Query (b) mengikuti arah relasi `Visitor -> Item` dan memakai `count(DISTINCT ...)` untuk menghitung jumlah visitor dan item unik pada setiap fold. Query (c) mengelompokkan relasi berdasarkan `type(r)` untuk melihat jumlah view, add-to-cart, dan transaksi.

Hasil query (a):

Jenis simpul	Jumlah
Visitor	1.083.222
Item	210.050
Total	1.293.272

Hasil query (b) dan (c):

Fold	Visitor	Item	Relasi	VIEWED	ADDED_TO_CART	BOUGHT
P1	884.614	192.603	1.735.130	1.678.507	42.437	14.186
P2	215.792	93.162	378.561	365.419	10.004	3.138

Jumlah visitor dan item unik dari Neo4j sama dengan hasil pada CSV di no. 2, sehingga filter pada CSV dan pada graf sudah konsisten. Dari tabel juga terlihat bahwa sebagian besar relasi berupa `VIEWED` (sekitar 96,7%), sedangkan transaksi hanya sekitar 0,8%.

2.1.4 VSM terhadap Simpul User (Visitor)

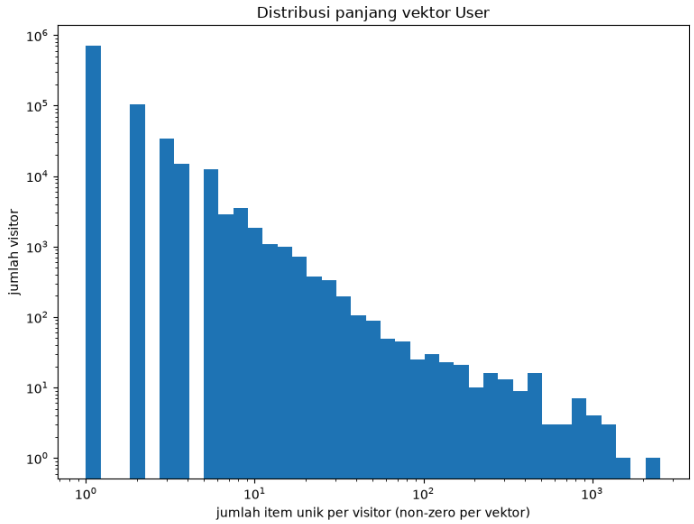
Pada VSM User, setiap visitor direpresentasikan sebagai vektor yang dimensinya sama dengan jumlah item pada P1. Nilai setiap elemen adalah jumlah bobot interaksi visitor tersebut terhadap item yang bersangkutan.

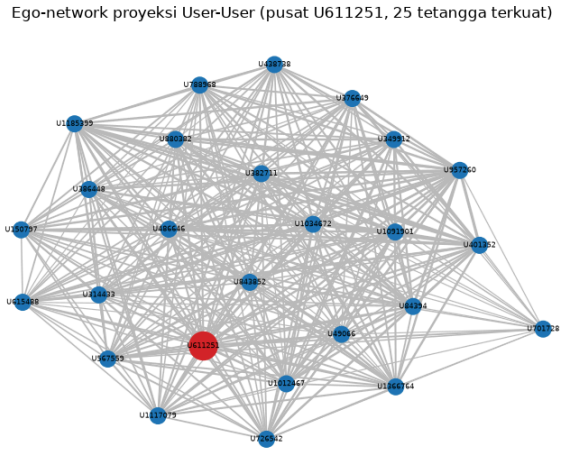
```
\vec{u} = (w_{u,1}, w_{u,2}, \dots, w_{u,|I|}), \quad w_{u,i} = \sum
\text{bobot event visitor } u \text{ pada item } i
```

Matriks yang terbentuk berukuran 884.614 × 192.603. Jika disimpan dalam bentuk dense, matriks ini membutuhkan sekitar 681,5 GB memori, sehingga digunakan sparse matrix format CSR dari `scipy.sparse` yang hanya membutuhkan 14,3 MB. Proyeksi User dibuat dalam bentuk graf User–User (one-mode projection), yaitu dua visitor dihubungkan jika pernah berinteraksi dengan item yang sama. Bobot edge-nya adalah jumlah item yang sama, dihitung dengan $B \cdot B^{T}$ di mana B adalah versi biner dari matriks VSM.

```
ui = train.groupby(["visitorid", "itemid"], as_index=False)["weight"].sum()
user_index = pd.Index(np.sort(ui["visitorid"].unique()))
item_index = pd.Index(np.sort(ui["itemid"].unique()))
X = sp.csr_matrix((ui["weight"].astype(np.float32),
                   (user_index.get_indexer(ui["visitorid"]),
item_index.get_indexer(ui["itemid"]))),
                  shape=(len(user_index), len(item_index)))
B = X.copy(); B.data[:] = 1          # versi biner untuk proyeksi
P_user = (B @ B.T).tocsr()           # proyeksi User-User
P_user.setdiag(0); P_user.eliminate_zeros()
```

Beberapa contoh vektor User (hanya elemen yang tidak nol) adalah U5 = {61396: 1}, U7 = {139394: 1, 164941: 1, 226353: 1}, dan U13 = {278689: 1, 320589: 1}.





Gambar kiri menunjukkan distribusi jumlah item per visitor dalam skala log. Sekitar 80% visitor hanya memiliki 1 elemen tidak nol, dengan median 1 item dan maksimum 2.528 item. Visitor dengan ribuan item kemungkinan besar adalah bot atau crawler. Gambar kanan adalah ego-network dari visitor U611251 bersama 25 tetangga terkuatnya (26 simpul dan 315 edge). Visitor pusat dipilih dari visitor yang paling banyak tetangganya di antara visitor yang menyentuh paling banyak 20 item, supaya tidak mengambil akun bot. Secara keseluruhan proyeksi User menghasilkan graf yang sangat besar, yaitu 37,1 juta edge, dan banyak di antaranya terbentuk hanya karena dua visitor kebetulan melihat satu item populer yang sama.

2.1.5 VSM terhadap Simpul Item

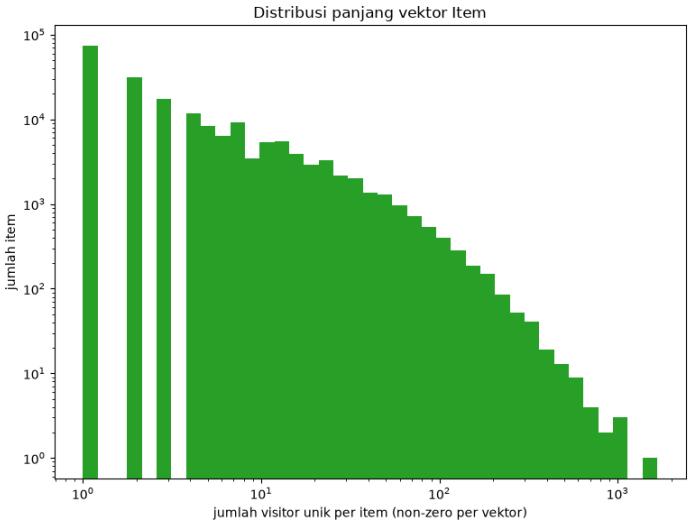
Pada VSM Item, setiap item direpresentasikan sebagai vektor yang dimensinya sama dengan jumlah visitor pada P1. Matriks ini merupakan transpos dari VSM User, sehingga ukurannya 192.603 × 884.614 dengan jumlah elemen tidak nol yang sama, yaitu 1.347.582.

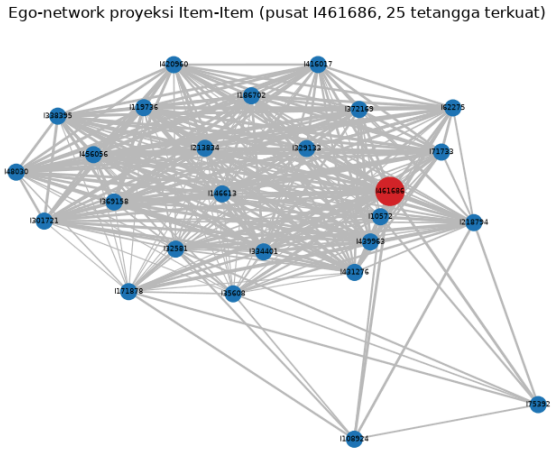
```
\vec{i} = (w_{1,i}, w_{2,i}, \dots, w_{|U|,i})
```

Proyeksi Item dibuat dalam bentuk graf Item–Item, yaitu dua item dihubungkan jika pernah disentuh oleh visitor yang sama ($B^T \cdot B$).

```
X_item = X.T.tocsr()                # matriks Item x User
B_item = B.T.tocsr()
P_item = (B_item @ B_item.T).tocsr() # proyeksi Item-Item
P_item.setdiag(0); P_item.eliminate_zeros()
```

Beberapa contoh vektor Item adalah I4 = {1042455: 1}, I9 = {949658: 1, 1205411: 1}, dan I15 = {88808: 1, 232847: 1, ..., 724401: 2, ...}.





Dibandingkan VSM User, vektor item lebih padat. Rata-rata satu item disentuh oleh sekitar 7 visitor (median 2). Ego-network pada gambar kanan berpusat pada item I461686, yaitu item dengan total bobot interaksi terbesar di P1 (690 visitor unik), bersama 25 tetangga terkuatnya (26 simpul dan 286 edge). Terlihat bahwa tetangga-tetangganya juga saling terhubung, sehingga membentuk kelompok produk yang mirip.

2.1.6 Perbandingan Proyeksi

Perbandingan karakteristik kedua proyeksi ditunjukkan pada tabel berikut.

Karakteristik	VSM / Proyeksi User	VSM / Proyeksi Item
Shape VSM	884.614 × 192.603	192.603 × 884.614
Elemen tidak nol	1.347.582	1.347.582
Density VSM	0,000008	0,000008
Rata-rata elemen tidak nol per vektor	1,52	7,00
Median elemen tidak nol per vektor	1	2
Maksimum elemen tidak nol per vektor	2.528	1.666
Vektor dengan 1 elemen tidak nol	79,8%	38,2%
Memori sparse VSM	14,3 MB	11,6 MB

Karakteristik	VSM / Proyeksi User	VSM / Proyeksi Item
Simpul proyeksi	884.614	192.603
Simpul terisolasi	5,2%	36,9%
Edge proyeksi	37.059.433	16.467.332
Rata-rata degree	83,8	171,0
Density proyeksi	0,000095	0,000888
Memori proyeksi	596,5 MB	264,2 MB
Waktu hitung	2,36 detik	0,74 detik

Berdasarkan tabel tersebut, proyeksi Item lebih baik dibandingkan proyeksi User, baik dari segi efisiensi maupun efektivitas.

Dari segi efisiensi, jumlah simpul pada proyeksi Item sekitar 4,6 kali lebih sedikit (193 ribu dibanding 885 ribu) dan jumlah edge-nya kurang dari setengah proyeksi User (16,5 juta dibanding 37,1 juta). Memori dan waktu komputasinya juga lebih kecil. Selain itu, similarity antar-item cukup dihitung satu kali lalu bisa dipakai untuk semua visitor, sedangkan visitor terus bertambah dan berganti dari waktu ke waktu.

Dari segi efektivitas, vektor item lebih padat (rata-rata 7,0 elemen tidak nol dibanding 1,5 pada vektor user). Karena sekitar 80% vektor user hanya berisi 1 elemen, nilai cosine similarity antar-user hampir selalu 0 atau 1 sehingga kurang bisa membedakan kemiripan. Proyeksi User memang memiliki simpul terisolasi yang lebih sedikit (5% dibanding 37%), tetapi sebagian besar user terhubung hanya karena melihat satu item populer yang sama, sehingga hubungan tersebut kurang menggambarkan preferensi. Simpul Item yang terisolasi umumnya adalah item yang hanya disentuh oleh 1 visitor, sedangkan sekitar 122 ribu item lainnya memiliki tetangga dengan bobot co-occurrence yang lebih bermakna. Preferensi terhadap item juga cenderung lebih stabil dibanding perilaku visitor anonim, sehingga lebih cocok untuk memprediksi interaksi pada P2.

Oleh karena itu, perhitungan similarity pada no. 7 dan KNN pada bagian Analytics dilakukan menggunakan proyeksi Item.

2.1.7 Perhitungan Similarity pada Graf Terproyeksi

Similarity antar-item dihitung menggunakan cosine similarity pada vektor item yang sudah diberi bobot.

```
\text{sim}(a,b) = \frac{\vec{a}\cdot\vec{b}}
{\lVert\vec{a}\rVert\,\lVert\vec{b}\rVert} = \frac{\sum_u w_{u,a}\,w_{u,b}}
{\sqrt{\sum_u w_{u,a}^2}\,\sqrt{\sum_u w_{u,b}^2}}
```

Setiap vektor item terlebih dahulu dinormalisasi L2, sehingga matriks similarity untuk semua pasangan item bisa didapat dengan satu kali perkalian matriks sparse $S = X \cdot X^{\top}$. Proses ini memakan waktu 0,8 detik dan menghasilkan 16.467.332 pasangan item dengan nilai similarity tidak nol, yang posisinya sama dengan edge pada proyeksi Item–Item. Record acuan yang dipilih adalah item 461686, yaitu item dengan total bobot interaksi terbesar di P1 (690 visitor unik).

```
X_item_norm = normalize(X_item, norm="l2", axis=1)
S = (X_item_norm @ X_item_norm.T).tocsr()
S.setdiag(0); S.eliminate_zeros()

def top_similar(pos, k=5):
    row = S.getrow(pos)
    order = np.lexsort((item_index[row.indices], -row.data))[:k]  # skor
menurun, itemId menaik
    return pd.DataFrame({"itemId": item_index[row.indices[order]],
                         "similarity": row.data[order].astype(float)})
```

(a) CYPHER untuk 5 record dengan similarity tertinggi

Nilai similarity yang sama juga dihitung langsung di Neo4j dari graf bipartit dengan hanya memakai relasi `fold = 'P1'`. Query ini selesai dalam 1,1 detik.

```
// 1. Norma vektor item acuan
MATCH (ref:Item {itemId: $ref})<-[r:VIEWED|ADDED_TO_CART|BOUGHT]-(v:Visitor)
WHERE r.fold = 'P1'
WITH ref, v, sum(r.weight) AS wa
WITH ref, collect({v: v, wa: wa}) AS vecA, sqrt(sum(wa * wa)) AS normA
// 2. Dot product dengan item lain yang disentuh visitor yang sama
UNWIND vecA AS a
WITH ref, normA, a.v AS v, a.wa AS wa
MATCH (v)-[r2:VIEWED|ADDED_TO_CART|BOUGHT]->(other:Item)
WHERE r2.fold = 'P1' AND other <> ref
WITH ref, normA, other, v, wa, sum(r2.weight) AS wb
WITH ref, normA, other, sum(wa * wb) AS dot
// 3. Norma vektor item kandidat
CALL (other) {
```

```
  MATCH (other)<-[r3:VIEWED|ADDED_TO_CART|BOUGHT]-(u:Visitor)
  WHERE r3.fold = 'P1'
  WITH u, sum(r3.weight) AS wb
  RETURN sqrt(sum(wb * wb)) AS normB
}
RETURN ref.itemId AS source, other.itemId AS target, dot / (normA * normB) AS
similarity
ORDER BY similarity DESC, target ASC
LIMIT 5
```

Pertama, `sum(r.weight)` untuk setiap visitor membentuk elemen vektor item acuan, lalu `normA` dihitung sebagai panjang vektor tersebut. Kedua, dari setiap visitor tadi dicari item lain yang juga disentuh pada P1, kemudian `sum(wa * wb)` menghasilkan dot product karena hanya visitor yang berinteraksi dengan kedua item yang ikut dihitung. Ketiga, subquery `CALL (other)` menghitung norma vektor setiap item kandidat, lalu dot product dibagi dengan hasil kali kedua norma.

Rank	source	target	similarity
1	461686	218794	0,378314
2	461686	171878	0,368817
3	461686	32581	0,345948
4	461686	50254	0,259665
5	461686	63338	0,259665

Hasil dari Cypher sama dengan hasil perhitungan Python, yang dicek dengan `np.array_equal` dan `np.allclose` (keduanya bernilai `True`). Item yang paling mirip dengan item 461686 adalah item 218794 dengan nilai similarity 0,3783.

(b) Menulis nilai similarity kembali ke graf

Kelima nilai similarity tersebut ditulis ke graf sebagai relasi baru `(:Item)-[:SIMILAR_TO {score, method, fold}]->(:Item)`. Sebelum ditulis, relasi `SIMILAR_TO` yang lama dihapus terlebih dahulu dengan `MATCH ()-[s:SIMILAR_TO]->() DELETE s` supaya hasilnya tetap sama jika sel dijalankan ulang.

```
// tulis similarity
UNWIND $rows AS row
MATCH (a:Item {itemId: $ref}), (b:Item {itemId: row.target})
MERGE (a)-[s:SIMILAR_TO]->(b)
SET s.score = row.similarity, s.method = 'cosine', s.fold = 'P1'

// tampilkan hasil
MATCH (a:Item {itemId: $ref})-[s:SIMILAR_TO]->(b:Item)
RETURN a.itemId AS source, b.itemId AS target, s.score AS similarity,
s.method AS method, s.fold AS fold
ORDER BY s.score DESC, target ASC
```

Jumlah relasi `SIMILAR_TO` berubah dari 0 menjadi 5. Hasil query-nya di Neo4j adalah sebagai berikut.

source	target	similarity	method	fold
461686	218794	0,378314	cosine	P1
461686	171878	0,368817	cosine	P1
461686	32581	0,345948	cosine	P1
461686	50254	0,259665	cosine	P1
461686	63338	0,259665	cosine	P1

2.1.8 Kondisi Graf Sebelum dan Sesudah Similarity

Untuk melihat perbedaan kondisi graf, digunakan query berikut di Neo4j Browser. Query sebelum hanya menampilkan relasi interaksi, sedangkan query sesudah ditambah relasi `SIMILAR_TO`.

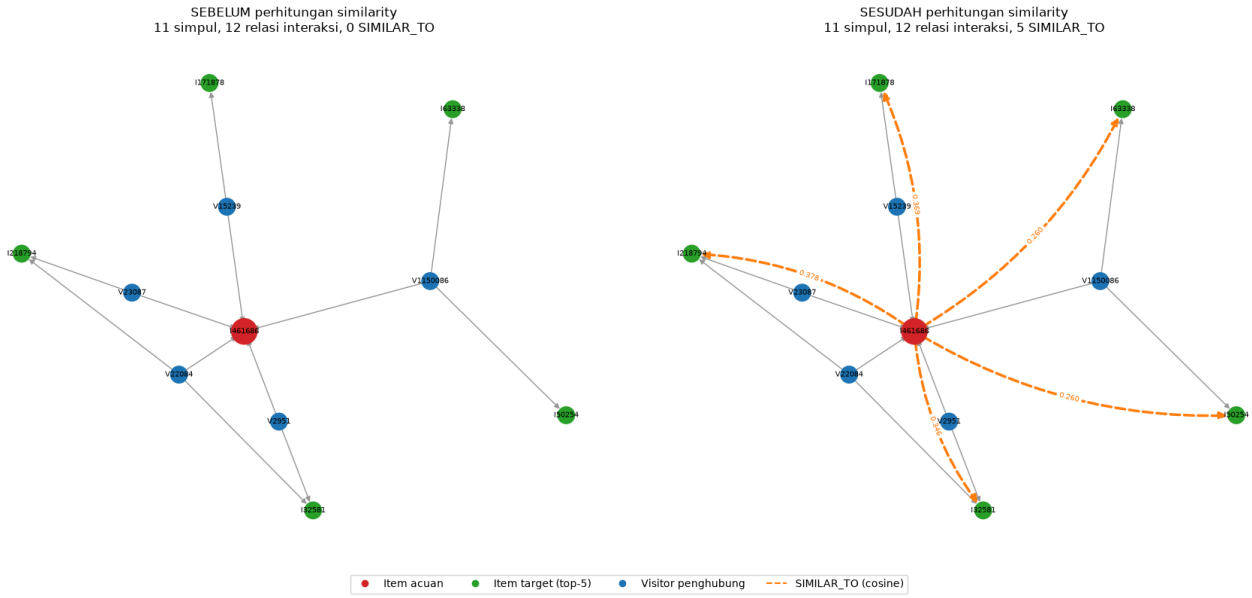
```
// SEBELUM: hanya relasi interaksi Visitor -> Item
MATCH p = (ref:Item {itemId: 461686})<-[r1]-(v:Visitor)-[r2]->(o:Item)
WHERE r1.fold = 'P1' AND r2.fold = 'P1'
  AND o.itemId IN [218794, 171878, 32581, 50254, 63338]
RETURN p LIMIT 30

// SESUDAH: ditambah relasi SIMILAR_TO
MATCH p = (ref:Item {itemId: 461686})<-[r1]-(v:Visitor)-[r2]->(o:Item)
WHERE r1.fold = 'P1' AND r2.fold = 'P1'
  AND o.itemId IN [218794, 171878, 32581, 50254, 63338]
RETURN p LIMIT 30
UNION
MATCH p = (:Item {itemId: 461686})-[:SIMILAR_TO]->(:Item)
RETURN p
```

Di Python, query yang dipakai terdiri dari tiga bagian yang digabung dengan `UNION`. Bagian pertama mengambil relasi visitor penghubung ke item acuan, bagian kedua mengambil relasi visitor penghubung ke item target (maksimal 3 visitor per item), dan bagian ketiga mengambil relasi `SIMILAR_TO` yang hanya muncul jika relasinya sudah ditulis.

Kondisi	Simpul	Relasi interaksi	Relasi SIMILAR_TO
Sebelum	11 (1 item acuan, 5 item target, 5 visitor)	12	0
Sesudah	11	12	5

Visitor penghubung yang muncul adalah 2951, 22084, 15239, 23087, dan 1150086. Semuanya memiliki relasi `VIEWED` ke item acuan, dan visitor 22084 juga memiliki relasi `ADDED_TO_CART` ke item 32581.



Sebelum similarity dihitung, hubungan antara item acuan dan kelima item target hanya bisa dilihat secara tidak langsung melalui visitor yang sama (jalur dua langkah `Item <- Visitor -> Item`). Setelah similarity ditulis, hubungan tersebut menjadi relasi langsung `SIMILAR_TO` yang memiliki nilai `score`. Dengan begitu, item yang mirip bisa langsung diambil dengan `MATCH (:Item {itemId: 461686})-[s:SIMILAR_TO]->(rec) RETURN rec ORDER BY s.score DESC` tanpa perlu menghitung ulang.

2.2.1 Rekomendasi KNN untuk User Acak

Pemilihan user dilakukan secara acak tetapi tetap bisa direproduksi dengan langkah berikut.

1. Kandidat user dibatasi pada visitor yang aktif di P1 dan P2 (supaya bisa dievaluasi) serta memiliki minimal 5 item unik di P1 (supaya riwayatnya cukup untuk KNN). Jumlah kandidat yang memenuhi syarat adalah 2.232 visitor.
2. Kandidat diurutkan berdasarkan `visitorId`, kemudian diambil satu visitor menggunakan `numpy.random.default_rng(seed=42)`. Karena seed dan urutannya tetap, hasil yang didapat akan selalu sama setiap kali dijalankan.

```
SEED = 42
MIN_ITEMS_P1 = 5
items_p1 = train.groupby("visitorid")["itemid"].nunique()
pool = np.sort(np.intersect1d(items_p1[items_p1 >= MIN_ITEMS_P1].index,
                              test["visitorid"].unique()))
rng = np.random.default_rng(SEED)
TARGET_USER = int(rng.choice(pool))   # -> 132066
```

User yang terpilih adalah visitor 132066 dengan riwayat 75 item pada P1. Item dengan bobot terbesar pada riwayatnya antara lain 165374 (bobot 20, addtocart dan view), 250212 (17), 241555 (15), 175885 (14), dan 122069 (9, transaction dan view).

(a) Skrip Python KNN (item-based)

Sesuai hasil no. 6, KNN dijalankan pada proyeksi Item. Untuk setiap item i pada riwayat user diambil k = 5 tetangga terdekat $N_k(i)$ berdasarkan nilai cosine similarity pada matriks S. Skor setiap item kandidat j kemudian dihitung dengan rumus berikut.

```
\text{score}(j) = \sum_{i \in I_u,\; j \in N_k(i)} w_{u,i} \cdot \text{sim}
(i, j)
```

Dengan rumus ini, nilai similarity tetangga dikalikan dengan bobot interaksi user pada item asalnya, sehingga item yang berasal dari interaksi kuat (misalnya add to cart) mendapat skor lebih tinggi. Tiga item dengan skor tertinggi dipilih sebagai item yang diiklankan.

```
def knn_recommend(visitor_id, k=5, n=3):
    user_row = X.getrow(user_index.get_loc(visitor_id))
    scores, sources = {}, {}
    for item_pos, w in zip(user_row.indices, user_row.data):
        neighbors = S.getrow(item_pos)
        order = np.lexsort((item_index[neighbors.indices], -neighbors.data))
[:k]
        for nb_pos, sim in zip(neighbors.indices[order],
neighbors.data[order]):
            nb_item = int(item_index[nb_pos])
            scores[nb_item] = scores.get(nb_item, 0.0) + float(w) *
float(sim)
            sources.setdefault(nb_item, []).append(int(item_index[item_pos]))
    ranking = sorted(scores.items(), key=lambda x: (-x[1], x[0]))[:n]
    return pd.DataFrame(
        [(rank, item, score, sources[item]) for rank, (item, score) in
enumerate(ranking, start=1)],
        columns=["rank", "itemId", "score",
"tetangga_dari_item"]).set_index("rank")

recommendations = knn_recommend(TARGET_USER, k=5, n=3)
```

Item yang sudah pernah dilihat user tidak dihapus dari kandidat, karena dalam iklan e-commerce menampilkan kembali item yang pernah dilihat (retargeting) adalah hal yang umum. Walaupun begitu, sebuah item hanya bisa masuk ke rekomendasi jika ia merupakan tetangga dari item lain di riwayat user.

(b) 3 item yang diiklankan (k = 5)

Rank	itemId	Skor	Tetangga dari item (sebagian)	Sudah disentuh di P1
1	66466	79,405	39242, 54973, 85416, 111094, 117429, 122069, …	Ya
2	144542	78,363	39242, 54973, 66466, 85416, 111094, 117429, …	Ya
3	165374	55,966	39242, 54973, 66466, 111094, 117429, 122069, …	Ya

Ketiga item yang diiklankan adalah 66466, 144542, dan 165374. Ketiganya sudah pernah disentuh user pada P1, karena item-item di riwayat user saling bertetangga dekat pada proyeksi Item.

2.2.2 Evaluasi Hit@5 dan NDCG@5

Data uji (ground truth) yang dipakai adalah item yang disentuh user pada P2 dengan jenis event apa pun. Relevansi dibuat biner, yaitu rel = 1 jika item rekomendasi terdapat di data uji dan rel = 0 jika tidak.

```
\text{Hit@5} = \begin{cases}1 & \text{jika ada item relevan di 5 posisi teratas}\\0 & \text{lainnya}\end{cases}
```

```
\text{DCG@5} = \sum_{i=1}^{5} \frac{rel_i}{\log_2(i+1)}, \quad \text{IDCG@5} = \sum_{i=1}^{\min(|T|,5)} \frac{1}{\log_2(i+1)}, \quad \text{NDCG@5} = \frac{\text{DCG@5}}{\text{IDCG@5}}
```

```
def hit_at_k(recommended, truth, k=5):
    return int(any(item in truth for item in recommended[:k]))

def ndcg_at_k(recommended, truth, k=5):
    dcg = sum(1 / np.log2(i + 2) for i, item in enumerate(recommended[:k]) if item in truth)
    idcg = sum(1 / np.log2(i + 2) for i in range(min(len(truth), k)))
    return dcg / idcg if idcg > 0 else 0.0
```

Pada P2, visitor 132066 hanya menyentuh 1 item, yaitu item 465211. Daftar yang dievaluasi adalah 3 item iklan dari no. 1(b), sedangkan posisi 4 dan 5 dianggap kosong (rel = 0).

Rank	itemId	rel	$1/\log_2(\text{rank}+1)$	Kontribusi DCG
1	66466	0	1,0000	0
2	144542	0	0,6309	0
3	165374	0	0,5000	0
4	-	0	0,4307	0
5	-	0	0,3869	0

(a) Hit@5

Tidak ada item rekomendasi yang terdapat pada data uji {465211}, sehingga nilai Hit@5 = 0.

(b) NDCG@5

Nilai DCG@5 = 0. Karena jumlah item pada data uji hanya 1, nilai IDCG@5 = $1/\log_2(2) = 1$. Dengan demikian NDCG@5 = 0 / 1 = 0.

Sebagai pembanding, metrik yang sama juga dihitung untuk seluruh 2.232 visitor kandidat dengan k = 5 dan 3 item iklan. Hasilnya, rata-rata Hit@5 = 0,0762 dan rata-rata NDCG@5 = 0,0326. Jadi nilai 0 pada user terpilih masih sejalan dengan performa model secara umum.

Nilai yang rendah ini dapat dijelaskan dari beberapa hal. Ketiga item iklan merupakan item yang sudah pernah disentuh user di P1, sedangkan pada P2 user justru berpindah ke item lain. Data RetailRocket juga sangat sparse (sekitar 80% visitor hanya menyentuh 1 item) dan periode uji hanya 3 minggu, sehingga item uji untuk setiap user sangat sedikit. Selain itu, nilai NDCG@5 dibatasi oleh panjang daftar rekomendasi yang hanya 3 item. Hasil ini kemungkinan bisa ditingkatkan dengan menambah jumlah item iklan, memperbesar nilai k, atau menyaring item yang sudah pernah dilihat user.

Kesimpulan

Berdasarkan hasil yang diperoleh, dapat disimpulkan beberapa hal berikut.

1. Subgraph P1 dan P2 terdiri dari 1.083.222 simpul Visitor dan 210.050 simpul Item, dan jumlah ini sama antara data di Neo4j dan CSV hasil filter.
2. VSM User dan VSM Item sama-sama sangat sparse (density 0,000008), tetapi vektor item rata-rata 4,6 kali lebih padat. Proyeksi Item juga membutuhkan kurang dari setengah jumlah edge dan memori proyeksi User, sehingga proyeksi Item dipilih sebagai proyeksi yang lebih baik.
3. Perhitungan cosine similarity di Python dan di Cypher menghasilkan 5 item teratas yang sama untuk item 461686, dengan nilai tertinggi pada item 218794 (0,3783). Nilai-nilai ini sudah ditulis ke graf sebagai relasi `SIMILAR_TO`.
4. Rekomendasi KNN item-based (k = 5, 3 item iklan) untuk visitor 132066 menghasilkan Hit@5 = 0 dan NDCG@5 = 0. Pada seluruh 2.232 visitor kandidat, rata-rata Hit@5 = 0,0762 dan NDCG@5 = 0,0326.